

AI 寫的程式碼，工程師到底怎麼驗收？

一份關於 AI coding 品質保證實務的調查報告：現況數據、個人層的檢查流程、團隊層的 quality gate 設計、工具鏈地圖，以及可落地的 30/60/90 天路線圖。

主題 AI-assisted code quality assurance 資料期間 2025.01 – 2026.08 語言 繁體中文 · 英文術語保留原文

00 執行摘要

2026 年的軟體工程現場出現一個結構性的位移：**瓶頸已經從「寫程式」移到「驗收程式」**。AI 產出的速度遠超過人類審查的速度，於是所有品質保證的重量，都壓在 code review 與 CI/CD 這道閘門上。

調查歸納出五個核心發現：

- **採用率已飽和，信任度沒有跟上**。84–90% 的開發者在用 AI 寫程式，但只有 3% 「高度信任」它的正確性，46% 明確不信任。
- **「幾乎對，但不完全對」是頭號痛點**。66% 的開發者把它列為最大挫折來源——這正是最難靠測試抓出來、只能靠人讀懂的那一類錯誤。
- **吞吐量真的上升，穩定性真的下降**。多份 2026 年遙測資料顯示 throughput 提升的同時，code churn、incident rate、review time 全面惡化。
- **驗收正在往左右兩端分裂**。左端是 spec-driven 的事前約束（規格、context 檔、小批次），右端是自動化閘門（SAST / SCA / secrets / test gate）。中間那段「人工逐行讀」的容量已經是最稀缺資源。
- **有效的團隊不是「多 review」，而是「改變被 review 的東西」**。限制 PR size、要求 AI 產出標註、把可自動判定的規則全部前移，讓人只讀機器讀不出來的部分。

本報告的核心主張

AI 沒有改變品質保證的原理，它改變的是**經濟結構**：產出成本趨近於零，驗證成本沒有變。因此所有工程實務的調整方向只有一個——**降低單位驗證成本，並拒絕接受無法驗證的產出**。

01 現況：採用率飽和、信任落差擴大

先確立事實基礎。以下數據來自 2025–2026 年的大型開發者調查與工程遙測資料。

採用與信任

84% 開發者正在使用或計畫使用 AI 開發工具（前一年 76%） STACK OVERFLOW 2025	3% 「高度信任」AI 產出正確性的比例；46% 明確不信任 STACK OVERFLOW 2025
66% 把「幾乎對、但不完全對」列為最大挫折來源 STACK OVERFLOW 2025	45% 認為 debug AI 產出的程式碼比自己寫還花時間 STACK OVERFLOW 2025

這組數字本身就是報告的起點：**使用率跟信任度已經完全脫鉤**。開發者不是因為相信 AI 才用它，而是在「不太信、但還是要用」的狀態下工作——這正是驗收流程必須存在的理由。

速度的幻覺

METR 在 2025 年做了一項少見的randomized controlled trial：16 位資深開源開發者、246 個他們自己 repo 裡的真實 issue，隨機指派「可用 AI」或「不可用 AI」。

METR RCT · 2025

可用 AI 的組別完成時間**慢了 19%**。事前他們預期會快 24%；**即使實際變慢，事後他們仍然相信 AI 讓自己快了 20%**。

研究者本人強調樣本限於「大型成熟 repo 的資深維護者」，不可外推到所有情境。但這個「主觀感受與客觀量測背離」的現象，對驗收設計有直接意涵：**不能用開發者的自我回報來評估 AI 的品質效果**。

2026 年遙測：吞吐量上升、穩定性下降

兩份基於實際 git / PR 資料的 2026 年報告，指向同一個方向。

+861% 高 AI 採用期間的 code churn （短期內被改寫掉的程式碼） FAROS AI 2026 · 22,000 DEVS	+243% incidents-to-PR ratio：每個合併變更引發生產事故的 機率 FAROS AI 2026
+442% code review 中位時長增幅——資深工程師承接了品保負擔 FAROS AI 2026	+31% 更多 PR 未經 review 直接合併——閘門在量能壓力下被 繞過 FAROS AI 2026

LinearB 分析 810 萬個 PR、4,800 個團隊後的結果同樣鮮明：

指標	人類撰寫	AI 參與	意涵
30 天內合併率 merge rate	84.5%	32.7%	近三分之二的 AI PR 沒有走完流程
等待首次 review pickup time	約 200 分鐘	約 1,000 分鐘	審查者迴避大而不熟悉的變更
PR 規模 (P75)	157 行	400+ 行	超出人類單次審查的認知上限
重構佔比 refactor rate	37%	趨近於 0	AI 傾向新增路徑，而非改善既有程式

資料來源：LinearB 2026 Engineering Benchmarks (8.1M PRs / 4,800 teams / 42 countries)。「AI 參與」由工具遙測判定，非自我申報。

可維護性的長期趨勢

GitClear 分析 2023–2026 年間 6.23 億筆程式碼變更，觀察到結構性的劣化訊號：**區塊重複 (block duplication) 成長 81%**；**被搬移的程式碼 (moved code，重構的代理指標) 從 2022 年的 21% 掉到 2026 年的 3.8%**；copy/paste 佔比從 9.4% 升到 15.7%；掩蓋錯誤的寫法 (error-masking constructs，如空的 catch) 增加 47%。

換句話說：**開發者現在複製的機率是重構的約 5 倍**，而 2022 年是相反的。這不是 AI 的技術缺陷，而是驗收流程的缺口——複製一段能動的程式碼，比說服自己去重構要容易得多。

安全面

45% AI 完成的編碼任務引入 OWASP Top 10 等級的安全缺陷 VERACODE 2025 · 100+ MODELS	86% XSS (CWE-80) 情境的防禦失敗率；log injection 為 88% VERACODE 2025
19.7% 程式碼樣本中至少含一個不存在的套件名 (package hallucination) SLOPSQUATTING RESEARCH	43% 同一個幻覺套件名在重複提問十次時每次都出現——可被攻擊者預先註冊 SLOPSQUATTING RESEARCH

Veracode 特別指出：**模型變大並沒有讓安全性變好**，語法正確率持續提升的同時，安全表現多年來幾乎沒有變化。這意味著安全驗收不能寄望於「換個更強的模型」，只能靠流程。

資料解讀的注意事項

上述遙測資料多來自工程效能廠商 (Faros、LinearB、GitClear)，有商業立場，且**相關不等於因果**——高 AI 採用的團隊也可能同時在承受其他壓力。METR 的 RCT 因果性最強但樣本僅 16 人。建議把這些數字當作**風險方向的指標**，而不是精確的效果量。

02 心智模型：從 author 變成 verifier

所有有效的驗收實務，都建立在同一個前提上：

誰按下 merge，誰就對那段程式碼負全責。
程式碼是誰寫的，在事故檢討會上不構成任何辯護。

兩個必須避開的認知陷阱

- 1. 自動化偏誤 automation bias。Thoughtworks 在 2025 年 11 月的 Technology Radar 把「Complacency with AI-generated code」放進 Hold 環：開發者在幾次好經驗之後會降低審查強度，而 AI agent 產生的變更集又越來越大，直接超出人的審查量能。
- 2. 產出的流暢度不等於正確性。AI 的程式碼看起來永遠是「有自信的、格式正確的、命名合理的」。人類審查者對這些訊號的直覺——「寫得這麼整齊應該沒問題」——在 AI 產出上完全失效。

先決定信任邊界 trust boundary

不是所有程式碼都需要同樣強度的驗收。實務上先分級，才不會把有限的審查力氣平均分配掉：

層級	典型情境	驗收強度
T0 拋棄式	一次性腳本、探索性原型、資料清理	能跑就好。vibe coding 在這層完全合理
T1 內部工具	內部儀表板、CI 腳本、開發工具	自動化閘門 + 快速人工掃視
T2 生產程式	對外服務、共用函式庫	完整流程：逐行 review + 測試 + 安全掃描
T3 高風險	認證授權、金流、個資、加密、遷移腳本	視為「不可信輸入」：雙人 review、威脅建模、必要時人工重寫

最常見的組織性錯誤，是讓 T0 的工作習慣滲透到 T2/T3。明確寫下分級並讓它出現在 PR 模板裡，比任何口頭提醒都有效。

03 個人層：日常怎麼檢查

個人層的驗收分成三段：事前約束、閱讀 diff、執行驗證。多數品質問題其實是在第一段就決定的。

3.1 事前約束：讓產出一開始就可驗收

- 先寫規格，再讓 AI 動手 (spec-driven development)。2026 年浮現的主流做法：先產出一份可審閱的規格／計畫 (plan mode、GitHub Spec Kit 之類的流程)，人審過規格再進入實作。審一頁規格的成本，遠低於審四百行 diff。

- **把專案慣例寫成 context 檔**。AGENTS.md / CLAUDE.md / .cursorrules 之類的檔案，用來固定架構慣例、命名規則、禁用的函式庫、測試方式。這是「context engineering」——把重複糾正的內容一次性外部化。
- **強制小批次 small batches**。DORA 2025 把「小批次作業」列為放大 AI 效益的關鍵能力之一。實務規則：一個 PR 對應一個可獨立描述的意圖，並設定行數上限（常見 200–400 行）。要求 AI 拆成多次提交，而不是一次生成完。
- **先問「這段該不該讓 AI 寫」**。認證、權限判斷、金流、加密、資料遷移——這幾類的錯誤成本遠高於節省的時間。

3.2 閱讀 diff：五個必查項目

關鍵原則：**讀 diff，不要讀 AI 對自己的說明**。模型的摘要傾向描述「它打算做什麼」，而不是「它實際改了什麼」。

☐ ① 這些 API 和套件真的存在嗎？

對照官方文件與 registry 檢查每一個新引入的相依套件——這是 package hallucination 與 slopsquatting 的唯一防線。實務做法：鎖定 lockfile、對 AI agent 的套件安裝設 allowlist、任何新相依都要人工核可。

☐ ② 邊界條件與錯誤處理對嗎？

空集合、null、超時、併發、重試、部分失敗。特別留意**掩蓋錯誤的寫法**（空的 catch、吞掉例外、萬用 try 包整段）——這類寫法在 AI 產出中增加了 47%，而且它們會讓錯誤延後到生產環境才爆。

☐ ③ 它是不是重造了已經存在的東西？

AI 看不到你整個 codebase 的全貌，最典型的失敗是複製一份既有工具函式的變體。搜尋關鍵字、確認有沒有現成實作，這是對抗 duplication 與 maintainability gap 最直接的動作。

☐ ④ 安全面站得住嗎？

輸入驗證、輸出跳脫（XSS）、SQL 參數化、authn/authz 檢查是否被略過、secrets 有沒有寫死、日誌有沒有記錄敏感資料、CORS 與權限預設值。Veracode 的資料顯示這是 AI 產出最系統性的弱點。

☐ ⑤ 它符合這個專案的做法嗎？

context blindness：分層架構被繞過、錯誤處理風格不一致、命名慣例不符、該用既有 client 卻自己開了新的 HTTP 呼叫。這類問題所有靜態工具都抓不到，是人工 review 最不可取代的部分。

一個有效的技巧：反向解釋

對 diff 中你不完全確定的部分，**要求自己（而不是 AI）用一句話說出它為什麼正確**。說不出來，就是還沒 review 完。這個動作直接對抗「幾乎對、但不完全對」——那 66% 的挫折來源，本質上是**看起來合理但邏輯有洞**的程式碼。

3.3 執行驗證：測試怎麼做才算數

- **一定要真的跑起來。**看起來對、也通過型別檢查的程式碼，仍然可能在第一次執行就失敗。手動測邊界情境，不要只信 happy path。
- **不要讓同一次對話同時產生實作與測試。**模型會寫出**迎合實作的測試**（tautological test）——實作有 bug，測試就跟著 bug 一起通過。做法：測試先寫（TDD），或由人定義測試案例、AI 只負責填實作。
- **用 mutation testing 檢驗測試本身。**覆蓋率只證明「這行被執行過」，不證明「這行錯了會被抓到」。在 AI 大量產生測試的環境下，coverage 已經是很弱的訊號，抽樣做突變測試才知道測試有沒有實質效力。
- **property-based testing 對 AI 產出特別有效。**AI 擅長讓具名的測試案例通過，卻常在隨機輸入下崩潰。定義不變量（invariant）讓工具去找反例。
- **對照既有行為。**重構類的變更，用舊實作與新實作跑同一批輸入比對輸出（differential testing），比讀 diff 可靠得多。

04 團隊層：把驗收固化成 quality gate

個人紀律無法規模化，而且會在交付壓力下第一個崩潰（Faros 觀察到的「未經 review 合併增加 31%」就是這個現象）。團隊層要做的是**把所有可自動判定的檢查前移，讓人只讀機器讀不出來的東西。**

4.1 六道關門

GATE 0	本機／提交前 format · lint · type check · secret scan · unit test
GATE 1	靜態分析 SAST · 複雜度 · 重複偵測 · 依賴新鮮度
GATE 2	測試關門 new-code coverage 門檻 · 抽樣 mutation testing · 契約測試
GATE 3	供應鏈與安全 SCA · lockfile 驗證 · 新相依人工核可 · SBOM · IaC scan
GATE 4	人工審查 架構意圖 · 業務邏輯 · 高風險路徑 · AI review bot 作為第一輪過濾
GATE 5	漸進式交付 feature flag · canary · 自動 rollback · 錯誤率監控

Gate 0–3 應該全自動且具阻斷力（blocking）；Gate 4 是唯一稀缺的人力資源，必須被前面幾道關門保護；Gate 5 承認一件事實——**總會有東西漏過去，所以要能快速偵測並回滾。**

4.2 針對 AI 產出的額外規則

政策	具體做法	解決的問題
產出標註 provenance	PR 模板加上「AI 參與程度」欄位（無／輔助／主要／agent 全自動）；commit 使用 trailer 標記	事故回溯、風險分層、指標統計的前提
PR 規模上限	超過門檻（如 400 行）自動標記並要求拆分	AI PR 是人類 PR 的 2.5 倍大，直接壓垮審查量能
禁止自動合併	agent 產出的 PR 一律需要人類核可，不得使用 auto-merge	對抗「未 review 合併增加 31%」
相依套件白名單	新套件需人工核可；CI 驗證套件存在時間與下載量	package hallucination / slopsquatting
新程式碼標準 clean as you code	對「本次變更的程式碼」設嚴格門檻，不追究舊債	讓門檻可執行，避免全域標準永遠達不到
高風險路徑加簽	authn/authz、金流、遷移腳本要求第二位審查者（CODEOWNERS）	把有限的資深人力放在事故成本最高處
review SLA	設定首次審查回應時限，並監測 pickup time	AI PR 等待首評時間是人類的 5 倍

一個關鍵的取舍

DORA 2025 的核心結論是「**AI 是放大器**」——它放大組織既有的優勢，也放大既有的弱點。但 Faros 2026 的遙測提出了一個更冷的觀察：**DevOps 成熟度高的組織，在高 AI 採用下的下游劣化幅度與低成熟度組織相當**。兩者的差異可能在於「調查自評」與「客觀遙測」的落差。合理的解讀是：**既有的良好實務是必要條件，但不是充分條件**；針對 AI 產出特性的新規則（PR 規模、標註、相依控管）必須另外加上。

05 工具鏈地圖

沒有單一工具能覆蓋 AI 產出的失效模式，實務上是分層疊加。以下按閘門分層列出代表性工具類別（為說明用途，非背書；請依自身技術棧選型）。

層	目的	代表工具類別	對 AI 產出的作用
Secrets	憑證外洩偵測	gitleaks · trufflehog · GitHub Secret Scanning	AI 常在範例中寫死 key
SAST	已知漏洞模式	Semgrep · CodeQL · SonarQube	抓 OWASP Top 10 類缺陷（Veracode 指出的 45% 主場）
SCA	相依套件風險	Snyk · Dependabot · OSV Scanner · FOSSA	幻覺套件、惡意套件、授權風險
IaC	基礎設施設定	Checkov · tfsec · Trivy	AI 生成的 Terraform / K8s 常有寬鬆預設值

層	目的	代表工具類別	對 AI 產出的作用
品質	重複、複雜度、覆蓋率	SonarQube (含 AI Code Assurance 類功能) • jscpd	直接對應 duplication +81% 的趨勢
測試強度	驗證測試本身	Stryker • PIT (mutation) • Hypothesis • fast-check (property-based)	破除 AI 生成測試的虛假覆蓋率
AI review	語意層審查	CodeRabbit • Greptile • Cursor Bugbot • GitHub Copilot code review • Qodo	作為第一輪過濾，處理 logic bug 與慣例偏離
交付安全	爆炸半徑控制	feature flag 平台 • canary / progressive delivery • 錯誤率監控	應對「漏網之魚必然存在」
工程遙測	量測驗收成效	DORA 指標平台 • PR 分析工具	提供 churn / incident / review 時間的客觀基線

用 AI 審查 AI 的正確定位

AI code review 工具能有效處理「機械性」的問題，並在大 PR 上提供第一輪過濾——但它**不能取代 Gate 4 的人類判斷**。理由很直接：判斷「這個變更是否符合我們的業務意圖與架構決策」需要模型沒有的組織脈絡，而且用同類模型審查同類模型，錯誤模式會相關 (correlated failure)。合理定位：**降低人類審查者的負載，而不是取代簽核責任**。

06 指標：怎麼知道驗收有效

不要用的指標

- acceptance rate (AI 建議採納率) —— 只衡量「按了 Tab」，與品質無關。
- 程式碼行數／PR 數量——在 AI 環境下這些數字必然上升，且與價值脫鉤。
- 開發者自評的生產力提升——METR 已經證明這個訊號會系統性地偏誤。
- 整體測試覆蓋率——AI 能輕易把覆蓋率拉高而不增加任何實質保護。

應該追蹤的指標

指標	觀察什麼	警訊
change failure rate	變更導致事故或需回滾的比例	與吞吐量同步上升
incidents per merged PR	單位變更的事故機率 (比事故總數更誠實)	任何持續上升
two-week code churn	兩週內被改寫掉的新程式碼比例	上升代表「先合併再說」
time to first review	PR 等待首次審查的時間	AI PR 顯著長於人類 PR
PRs merged without review	繞過閘門的比例	任何非零值都需要解釋
escaped defect rate	漏到生產環境的缺陷密度	驗收有效性的最終裁判

指標	觀察什麼	警訊
mutation score (抽樣)	測試實際偵錯能力	覆蓋率上升但突變分數下降
重複率與重構比例	可維護性趨勢	duplication 上升 + moved code 下降

把這些指標依「AI 參與程度」切分（這正是產出標註政策的價值所在），才能回答真正該問的問題：我們的驗收流程，對 AI 產出是否和對人類產出一樣有效？

07 落地路線圖

按投入成本與回報排序，不需要一次到位。

第 1–30 天 · 止血

- 公布一頁 AI usage policy：哪些場景可以用、哪些路徑（authn、金流、加密、遷移）需要額外審查、merge 責任歸屬。
- 開啟阻斷式的 secret scanning 與 SAST，先只擋 critical，避免一次噪音過大導致全員繞過。
- PR 模板加入「AI 參與程度」欄位，並設定 PR 行數上限的自動提醒。
- 關閉 agent 產出 PR 的 auto-merge。

第 31–60 天 · 建立基線與槓桿

- 導入 clean as you code：對新增／修改的程式碼設覆蓋率與品質門檻。
- 導入一套 AI review bot 作為第一輪過濾，明確定位為「減負」而非「簽核」。
- 建立 AGENTS.md / 規則庫，把重複糾正的架構慣例外部化。
- 建立指標基線：change failure rate、incidents per PR、churn、time to first review，並依 AI 參與程度切分。
- 新增相依套件納入人工核可流程，鎖定 lockfile。

第 61–90 天 · 結構性改變

- 推行 spec-driven 流程：先審規格、再產實作，把審查成本前移。
- 對關鍵模組導入抽樣 mutation testing 與 property-based testing。
- 建立 progressive delivery（feature flag + canary + 自動回滾），承認漏網之魚必然存在。
- 把「這個事故的程式碼是怎麼通過驗收的」列為事故檢討的固定問題，讓閘門持續演化。

常見反模式

- 只加 AI review bot，不改 PR 規模。大 PR 的問題是人的認知上限，不是缺乏意見。

- 用覆蓋率當唯一測試指標。在 AI 環境下這個指標最容易被無意義地滿足。
- 把所有規則一次設成阻斷式。噪音過大會導致團隊系統性地繞過閘門，比沒有閘門更糟。
- 依據吞吐量提升而裁減人力。下游的驗證、修復、維護工作並沒有消失，只是延後出現。
- 用自評問卷評估 AI 成效。METR 的結果顯示這個訊號不可靠。

08 附錄：合併前檢查清單

適用於 T2（生產程式）以上。可直接貼進 PR 模板。

- ☐ 我讀完了整個 diff，不是只讀了摘要
- ☐ 每個新引入的套件與 API 都經過查證確實存在
- ☐ 我能用一句話說明每個非顯而易見的決定為什麼正確
- ☐ 邊界條件已檢查：空值、超時、併發、部分失敗
- ☐ 沒有掩蓋錯誤的寫法（空 catch、吞例外、萬用 try）
- ☐ 沒有重造 codebase 裡已存在的功能
- ☐ 輸入驗證、輸出跳脫、authn/authz 檢查都在位
- ☐ 沒有寫死的 secrets，日誌沒有記錄敏感資料
- ☐ 符合本專案的架構與命名慣例
- ☐ 我實際執行過，並手動測過至少一個邊界情境
- ☐ 測試會因為實作有 bug 而失敗（不是迎合實作的測試）
- ☐ 這個 PR 的規模在可審查範圍內
- ☐ 我願意為這段程式碼在生產環境的行為負責

09 資料來源

- SURVEY [Stack Overflow 2025 Developer Survey — AI section](#)：採用率 84%、高度信任 3%、不信任 46%、「almost right」66%、debug 更費時 45.2%。

- **SURVEY** Google Cloud / DORA — 2025 State of AI-assisted Software Development：90% 使用率、AI 為「放大器」、七項 AI 能力模型、與交付穩定性的負向關係。
- **RCT** METR — Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity：16 位開發者、246 個 issue、慢 19%、自評快 20%。
- **TELEMETRY** Faros AI — The AI Engineering Report 2026：22,000 位開發者、churn +861%、incidents-to-PR +242.7%、review 時長 +441.5%、未 review 合併 +31.3%。
- **TELEMETRY** LinearB — 2026 Engineering Benchmarks：810 萬 PR、AI PR 合併率 32.7% vs 84.5%、pickup 5.25 倍、PR 規模 400+ 行。
- **CODE ANALYSIS** GitClear — The Maintainability Gap (2026)：6.23 億筆變更、duplication +81%、moved code 21% → 3.8%、error-masking +47%。
- **SECURITY** Veracode — 2025 GenAI Code Security Report：80 項任務、100+ 模型、45% 引入 OWASP Top 10 缺陷、XSS 86%、log injection 88%。
- **SECURITY** Cloud Security Alliance — Slopsquatting research note：223 萬樣本、19.7% 含幻覺套件、43% 幻覺名稱十次重測皆出現。
- **PRACTICE** Thoughtworks Technology Radar — Complacency with AI-generated code (Hold, 2025.11)：automation bias、變更集過大、建議以 TDD 與靜態分析對抗。
- **PRACTICE** Simon Willison — Vibe engineering：自動化測試、事前規劃、文件、版本控制、CI、code review 文化作為 AI 協作的前提。

調查報告 · 2026 年 8 月

資料期間 2025.01–2026.08。工程遙測資料來自商業廠商，具立場且相關不等於因果；
工具名稱僅為說明代表性類別，非背書。所有數據請以原始來源為準。

本文主要由 AI 工具研究與撰寫，未經完整人工查核；內容仍可能存在錯誤。

第三方資料均標注來源；工具名稱僅為識別與說明，非背書。

除另有標示外，本站原創內容採 CC BY 4.0；第三方素材不在本站授權範圍內。[回到索引](#)